

目 录

第一章 前截断章节	1
1.1 当前版本编译时间	1
第二章 绪论	2
2.1 研究背景与意义	2
2.2 研究目标	2
2.3 研究所面临的挑战	2
2.4 研究内容	2
2.5 论文主要工作与创新点	2
2.6 论文组织结构	2
第三章 相关技术及相关研究综述	3
3.1 路线搜索和推荐	3
3.2 本章小结	3
第四章 动态路网中的交通拥堵缓解：面向行程请求流的持续最优路线组合	4
4.1 引言	4
4.2 研究背景及应用场景	4
4.2.1 研究内容与挑战	5
4.2.2 研究成果与贡献	6
4.3 问题描述	6
4.3.1 Dijkstra 算法	6
4.3.2 位置点间的距离计算	6
4.3.3 问题定义	6
4.3.4 路网及其构成	6
4.4 精确遍历算法	7
4.5 自感知的批处理算法	7
4.5.1 初始路线搜索	8
4.5.2 批优化处理	10
4.5.3 CORC 问题的总体解决方案	13
4.6 实验研究	14
4.6.1 实验设定	14
4.6.2 实验结果	16

4.7 本章小结	20
第五章 联合移动模式挖掘	21
5.1 引言	21
5.1.1 研究背景与意义	21
5.1.2 研究内容与挑战	21
5.1.3 研究成果与贡献	21
5.2 问题描述	21
5.2.1 路网及其构成	21
5.2.2 Dijkstra 算法	21
5.2.3 位置点间的距离计算	21
5.2.4 问题定义	21
5.3 实验研究	21
5.4 本章小结	21
第六章 时空接触搜索	22
6.1 引言	22
6.1.1 研究背景与意义	22
6.1.2 研究内容与挑战	22
6.1.3 研究成果与贡献	22
6.2 问题描述	22
6.2.1 路网及其构成	22
6.2.2 Dijkstra 算法	22
6.2.3 位置点间的距离计算	22
6.2.4 问题定义	22
6.3 实验研究	22
6.4 本章小结	22
第七章 后截断章节	23
致 谢	24
参考文献	25
攻读博士学位期间取得的成果	27

图目录

图 4-1	行程请求和路线	4
图 4-2	行程请求数目 $ Q $ 的影响	16
图 4-3	行程请求到达速率的影响	16
图 4-4	参数 α & β 的影响	17
图 4-5	批优化时间间隔 T 的影响	17
图 4-6	参数 ϵ 和预检查策略的影响 (a) CPU 运行时间; (b) 通行时间; (a) 预 先检查策略的影响评估	19

表目录

表 4-1 参数设定.....	15
-----------------	----

主要符号表

第一章 前截断章节

1.1 当前版本编译时间

二零二六年一月二十八日 14:40

第二章 绪论

2.1 研究背景与意义

2.2 研究目标

2.3 研究所面临的挑战

2.4 研究内容

2.5 论文主要工作与创新点

2.6 论文组织结构

第三章 相关技术及相关研究综述

3.1 路线搜索和推荐

3.2 本章小结

第四章 动态路网中的交通拥堵缓解：面向行程请求流的持续最优路线组合

4.1 引言

4.2 研究背景及应用场景

随着基于位置服务的应用的兴起，路径规划服务已经成为我们生活中不可或缺的一部分。路径规划和行程推荐在近几年引起了学者们的广泛研究 [?????]。这些研究的研究目标是基于当前的交通状态为单行程制定最优的路线。值得关注的是，在路径规划服务使用越来越频繁的情况下，大量的用户很有可能会在极短的时间间隔内密集地发布行程请求，特别是在通勤时间等高峰时期，从而形成了持续的行程请求流。在这种新的场景下，实现针对行程请求流的路径规划这一需求变得更加迫切。已经存在的相关研究旨在为行程请求中的单个行程依次制定个人最优路线 [?]。然而，在为行程请求流规划路线时，仅仅基于当前的交通状态以个人最优为最终目标规划个人最优的路线，可能会导致交通拥堵。更合理的路线规划应该考虑到之前已经规划的行程会对未来的交通状态产生影响，因为它们会增加路网中各路段的车流量。

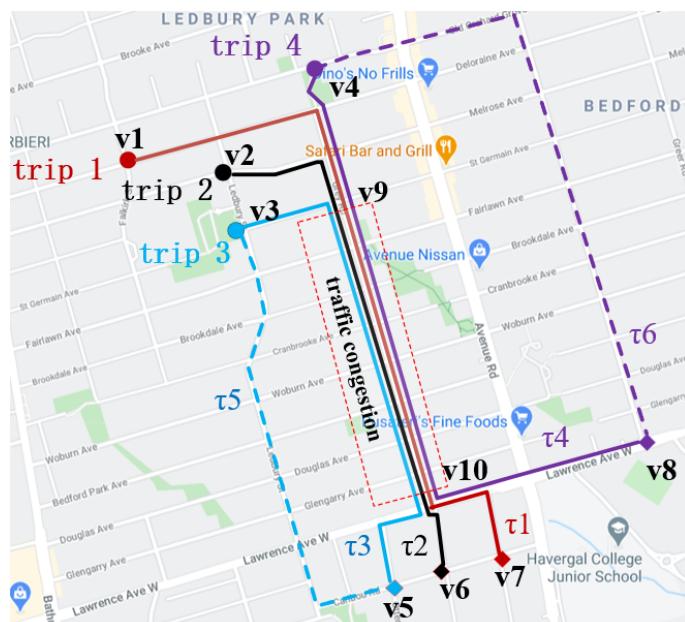


图 4-1 行程请求和路线

以图 1 为例，图中的每个行程包含一个起点和终点，分别用圆圈和钻石形状标记。顶点 $v_1 - v_4$ 是行程 1-4 对应的起点，而顶点 $v_5 - v_8$ 是行程 1-4 对应的终点。

行程请求 1–3 分别发布于 8:00:01, 8:00:01, 8:00:05。它们的最优路线分别是 τ_1, τ_2 和 τ_3 ，在图中我们用实线标记。这三条路线预计分别会在 $[8:02:00, 8:05:00]$, $[8:01:40, 8:04:40]$, $[8:01:30, 8:04:30]$ 行驶在路段 $e(v_9, v_{10})$ 。在 $[8:02:00, 8:04:30]$ ，它们都会都会行驶在路段 $e(v_9, v_{10})$ ，而这也许会导致交通拥堵，从而延路段 $e(v_9, v_{10})$ 所有正在行驶的其他车辆的通行时间。行程 4 发布于 8:00:20，基于在 8:00:20 这一时刻的交通状态我们为它制定的最优路线为 τ_4 。很明显，该规划没有意识到路段 $e(v_9, v_{10})$ 由于之前规划的路线 $\{\tau_1, \tau_2, \tau_3\}$ ，在未来可能会产生潜在的交通拥堵。糟糕的是，我们为行程 4 制定了最优路线 τ_4 ，该路线再一次经过了路段 $e(v_9, v_{10})$ ，也许会进一步加剧交通拥堵。综上，给定一组密集发布的行程请求流，为了缓解交通拥堵，我们需要把它们看作一个整体进行路线规划。如例 1，我们可以先把行程 1–3 看作一个整体，考虑到可能产生的交通拥堵。我们可以调整行程 3 的最优路线为 τ_5 ，在图 4-1 中用虚线表示。对当前到达的行程 4，通过把行程 1–4 看作一个整体，同理我们可以调整其最优路线为 τ_6 。这些调整后的路线对单个请求来说也许是次优的路线，但路段 $e(v_9, v_{10})$ 的交通拥堵却可以得到大大缓解，行程 1–4 总的通行时间也会大大减少。通过采用 $\{\tau_1, \tau_2, \tau_5, \tau_6\}$ 这一路线组合，行程 1–4 的总的通行时间可能会是最小的，我们称这样的一个路线组合是最优的路线组合。

4.2.1 研究内容与挑战

本文提出和研究了一种针对持续的行程请求流的路线规划问题：在一个动态路网上给定一组行程请求流，行程请求流中的行程请求数是不断增加的，我们需要找到一个路线组合 $\Pi = \{\pi_1, \pi_2, \dots\}$ 使得所有在 Π 中的路线的总的通行时间最小。其中，每一条路线 $\pi_i \in \Pi$ 是为每一个具体的行程请求 q_i 对应规划的。该问题具有十分广泛的应用场景，包括但不限于城市交通流量管理，实时的路线规划以及持续的拥堵预防上。针对该问题的精确算法具有指数级的时间复杂度，因此不能有效地运用到实际的应用场景中。为了解决这个问题，我们提出了一个自感知的批处理算法。对应的实验研究证明了该算法的准确性和高效性。

4.2.2 研究成果与贡献

4.3 问题描述

4.3.1 Dijkstra 算法

4.3.2 位置点间的距离计算

4.3.3 问题定义

4.3.4 路网及其构成

定义 4.1 (动态路网) 一个动态路网 $G=(V,E)$ 是由顶点集 V 和边集 E 构成的, 其中 $E \subseteq V \times V$, 顶点代表路段的连接点, 交叉点; 连边则代表具体的路段。每条边 $e(v_i, v_j) \in E$ 连接着两个顶点 v_i 和 v_j , 这里 $v_i, v_j \in V$ 。为了便于描述, 我们假定所有行程请求的起点和终点位置都已经被映射到离它们最近的顶点上。针对每个路段 e , 我们用 C_e 来代表该路段的车容量, 用 $T_m(e)$ 代表在路段 e 的最小通行时间 (e.g., 当路段 e 上没有其他车辆时的通行时间)。在我们的设定中, 每一个路段的权重 (通行时间) 是动态变化的。一个路段的最小通行时间是 $T_m(e)$, 而实际的通行时间是和该路段上的动态车流量成比例的。

定义 4.2 (通行时间函数) 车辆在时刻 t 经过路段 e 的通行时间和当前时刻路段 e 上的车流量 $f(e, t)$ 相关。参照已经存在的研究 [??], 基于当前时刻路段 e 上的车流量, 我们使用下式来计算路段 e 在时刻 t 的通行时间, 我们用 $T(e, t)$ 表示。

$$T(e, t) = T_m(e) \times (1 + \alpha \times (\frac{f(e, t)}{C_e})^\beta) \quad (4-1)$$

其中, C_e 代表路段 e 的车流量, α 和 β 是由路段的不同性质 (e.g., 限速, 路宽) 等决定的可变参数。我们的解决方案是和这些参数独立的, 如何为这些参数设置合适的值并不在我们的研究范围内。由式4-1我们可以知道, 随着车辆数目的动态变化, 每条路段的通行时间也会随之发生改变。

定义 4.3 (路线及其通行时间) 我们用一个有限的顶点序列 $\langle v_0, v_1, \dots, v_{|\pi|-1} \rangle$ 来表示一条路线 π 。假定路线 π 出发时间为 t_0 , t_i 是其经过的顶点 v_i 的预计到达时间 (i.e., $t_i = t_{i-1} + T(e(v_{i-1}, v_i), t_{i-1})$)。结合式4-1, 我们用式 $ET(\pi, t_0)$ 来计算一条路线 π 的预计通行时间, 即路线 π 中每个路段的预计通行时间之和, 我们用 $ET(\pi, t_0)$ 来表示。

$$ET(\pi, t_0) = \sum_{i=0}^{|\pi|-2} T(e(v_i, v_{i+1}), t_i) \quad (4-2)$$

之后我们使用 $\pi.v_0$ 和 $\pi.v_{|\pi|-1}$ 来分别代表一条路线 π 对应的第一个顶点 (起点) 和最后一个顶点 (终点)。

定义 4.4 (行程请求流) 我们用 $q = (s, d, t)$ 来代表一个行程请求, 其中 s 是该行程的起点位置, d 是该行程的终点位置, 而 t 是该行程的出发时间。 $Q = \{q_1, q_2, \dots\}$ 由多个行程请求组成, 代表一组行程请求流 (i.e., $\forall q_i, q_j \in Q, i > j: q_i.t \geq q_j.t$)。需要注意的是, Q 是以流数据的形式到达的行程请求集合并且 Q 是动态更新的。

定义 4.5 (全局最优路径规划问题) 在一个动态路网 $G = (V, E)$ 上, 给定一组行程请求流 $Q = \{q_1, q_2, \dots\}$ 和时间间隔 T , 持续的最优路线组合问题 (CORC 问题) 旨在不断地处理最近到达的一批行程请求 $Q_n = \{q_k, q_{k+1}, \dots, q_{|Q|}\}$ ($Q_n \subset Q \wedge q_{|Q|}.t - q_k.t = T$), 找到一个最优的路线组合 $\Pi = \{\pi_1, \pi_2, \dots\}$ 使得: (1) $|Q| = |\Pi|$; (2) $(\forall \pi_i \in \Pi) (\pi_i.v_0 = q_i.s \wedge \pi_i.v_{|\pi_i|-1} = q_i.d)$; (3) 在 Π 中规划的所有路线的总的通行时间最小, 我们用 $TT(\Pi)$ 来表示所有路线的总的通行时间 (参照公式4-3)。

$$TT(\Pi) = \sum_{i=1}^{|\Pi|} ET(\pi_i, q_i.t) \quad (4-3)$$

4.4 精确遍历算法

针对 CORC 问题, 一个简单直接的办法就是对每一个行程请求, 搜索起点和终点间可能存在的所有路线, 然后对所有的路线组合进行遍历评估, 比较各个路线组合所有路线总的通行时间, 最终筛选出具有最小通行时间的路线组合。具体来说, 对于每一个行程请求 $q_i \in Q$, 我们可以采取深度优先算法 (DFS) 来查找从起点 s_i 到终点 d_i 可能存在的所有路线。其中, 我们可以应用一些剪枝策略来加速深度优先搜索。随后, 对于每一个路线组合我们计算它所有路线总的通行时间。最终, 我们返回具有最小通行时间的路线组合作为返回结果。精确算法比较简单直接, 便于理解。当行程请求数量 $|Q|$ 较小时, 精确算法可以在有限时间内得到最优解。但是, 当行程请求数量 $|Q|$ 很大时, 精确算法显然大量的计算时间, 不能运用到实际的应用场景中。

4.5 自感知的批处理算法

为了高效解决 CORC 问题, 我们提出了一个自感知的批处理算法 (SBP 算法), 该算法主要包含两大组成部分: (1) 初始路线搜索和 (2) 批优化处理。具体来说, 给

定一组行程请求流 Q_n ，我们首先采用初始路线搜索生成一个高质量的初始路线组合 Π_n ，该组合包含了为该行程请求流中的每个请求指定的初始路线。接下来，我们执行批优化处理对初始路线组合中的每个初始路线进行优化。最终，我们将优化后的路线组合 Π_n 加入到全局的路线组合 Π 中去。每当我们处理完一批行程请求后，我们返回 Π 作为实时的规划结果。我们的自感知思想合理考虑了新规划的路线会影响未来的交通状态这一事实，即规划的路线会增加一些路段的车流量从而影响该路段的实际通行时间。

4.5.1 初始路线搜索

初始路线搜索的目标是为新一批到达的行程请求流中的每个行程请求规划一条高质量的个人路线。每个路段的动态车流量由两部分组成的，一部分是在我们规划系统中的请求车辆所造成的交通流量，即我们规划的路线在各路段上造成的车流量，我们称之为系统内的车流量，一部分是系统外的车流量即在各路段上没有使用我们规划系统的车辆数目，我们把系统外的车流量直接作为算法输入。

4.5.1.1 路段标签

为了实现对系统内车流量的快速计算，我们在每个路段上维持一系列的路段标签 $L_e = \{l_1, l_2, \dots\}$ 。这些路段标签记录了经过该路段的路线在该路段的时间信息。每个路段标签 $l_i = \{t_a, t_b\}$ 记录了一条特定的路线进入路段 e 的时间 t_a 以及离开该路段的时间 t_b 。刚开始没有行程请求的时候，每个路段维持的路段标签是一个空集。之后每当我们处理完一批新的行程请求集合的时候，对应路段的路段标签会进行动态地更新。

4.5.1.2 系统内的车流量计算

我们可以快速地计算在各个路段上由我们规划系统规划的路线所产生的车流量。由我们规划系统规划的路线在路段 e ，时刻 t 所造成的车流量，也就是满足 $[l_i.t_a, l_i.t_b] \ni t$ 的路段标签 l_i 的数量，其中 $l_i \in L_e$ 。在我们的实验中，我们应用了一个优先队列来加速计算。

4.5.1.3 通行时间下界

我们用一个启发值 $H(v_i, v_j)$ 来代表从一个顶点 v_i 到另一个顶点 v_j 的通行时间下界。给定最小的系统外的车流量作为静态的车流量输入，该下界是由 Dijkstra 算法 [?] 预计算的。在本文中，我们将非请求车辆的车流量直接作为算法输入。

4.5.1.4 算法描述

算法 4-1提出了初始路线搜索的伪代码。算法输入包含一个动态路网 $G = (V, E)$ ，一个行程请求 $q = (v_s, v_d, t)$ ，所有两点间的通行时间下界 $H(v_i, v_j)$ ，所有路段的路段标签集合 $\mathcal{L} = \{L_{e_1}, L_{e_2}, \dots\}$ 。算法输出为行程请求 q 制定的一条初始路线。每个顶点 $v \in V$ 包含以下记录信息：从起点 v_s 到达该点的精确通行时间 t_s ；从该点到达终点的通行时间下界 t_d ；从起点到达该点的最小时刻 et 以及一个前驱节点记录 $pred$ 。首先，我们初始化每一个顶点 $v \in V$ 的信息记录，设置路线 π 是一个空集 (line 1)。接着我们把起点 v_s 加入一个优先队列 PQ 中。该优先队列 PQ 内部对顶点对象按照该点的 $v.t_s + v.t_d$ 的值从小到大排序，队头元素为具有最小 $v.t_s + v.t_d$ 的顶点 (line 2)。在搜索过程中，每一次我们从 PQ 选出对头元素 v 并探索它的邻接顶点。具体来说，每一次我们选出的 v 都具有最小的 $v.t_s + v.t_d$ ，之所以这样选择，是基于这样的顶点扩张可能会产生最小的通行时间 (line 4)。如果选择的 v 是行程请求的终点 v_d ，我们就利用相应节点的前驱节点记录生成一条路线 π ，并将 π 作为结果返回 (lines 5–11)。如果选择的 v 不是行程请求的终点 v_d ，对于每一个从顶点 v 出发的路段 e ，我们计算当前时刻该路段的车流量 (line 13)。对于路段 $e(v, v')$ 的结束点 v' ，我们需要检查该点能否通过顶点 v 到达，并且所花的时间比从其他点到达该点更小 (line 14)。如果满足，我们将分别更新顶点 v' 对应的 t_s, et 和 $pred$ 记录 (lines 15–17)。接下来，如果顶点 v' 不在优先队列 PQ 中，我们将其加入。当优先队列 PQ 为空或者我们已经扩张到行程请求对应的终点时，初始路线搜索算法终止。

初始路线搜索：对于每一个新到达的行程请求（包含起点，终点，出发时刻），寻找一条在出发时刻出发，从起点能最快到达终点的路径（最短路径）。假设车辆通过特定路段的时间是和该路段的属性和车流量决定的，而车流量由两部分组成。一部分是非请求车辆车流量，即没有使用我们路线规划系统的车辆，直接作为输入数据，不是我们研究的范畴；一部分是请求车辆的车流量，即使用了我们规划系统的车辆，这些车辆按照规划的路线行驶而在经过的路段上增加的车流量。由于路网每个路段车流量是动态变化的，通行时间也会随之动态变化，我们就是要在这样一个动态变化的路网中寻找从出发时刻出发，起点到终点的最短路径。

初始路线搜索过程从起点开始，对与起点相连的邻接顶点做网络扩张，接着选择一个新的顶点继续扩张，直到扩张到终点。我们的扩张策略是，每次选择一个最可能最快到达终点的顶点作为下一个扩张顶点，什么是最可能最快到达终点的顶点呢？即（从起点到该点的精确耗时 + 从该点到终点的最小耗时）最小的顶点。从起点到该点的精确耗时是根据每个路段的实时车流量准确计算出来的，而从该

点到终点的最小耗时是预处理中计算的。在预处理中，我们取每个路段的最小车流量作为静态的车流量，利用 **dijkstra** 算法计算顶点两两之间的最短路径（即耗时最小的路径），并将顶点两两之间的最小耗时存储起来作为我们初始路线搜索的算法输入。综上，我们从起点开始，按照该扩张策略一直扩张，直到扩张到终点，则我们就找到了一条在出发时刻能最快到达终点的路径。

批优化处理：对于在较短时间间隔（比如 1s）内发出的行程请求集合（包含起点，终点，出发时刻），我们为这些请求分别调用初始路线搜索后，这些初始路线可能会导致潜在的拥堵，从而增加对应路段的通行时间，（比如说行程数量很多，而它们规划的路线都要经过同一路段，则可能造成拥堵）。因此这些最优路线的部分路段是需要优化的，我们将在同一时间间隔（比如 1s）内发出的行程请求作为一个整体，对我们之前为它们规划的初始路线集合进行批优化处理。对于一批初始路线集合，我们批优化策略如下：1）从该集合中选取其中一条路线，将该集合内的其他路线看作已经发布的路线（其实还没有发布），假定这些用户将会按照发布的路线行驶，对应道路上就会增加一定车流量，从而全局车辆（包括之前已经规划的车辆在每个路段的耗时也会变化）。我们需要计算这些路线在各个路段增加的车流量，更新路段上的车流量，更新所有行程在各路段的通行时间，这些就相当于更新了交通信息。2）对于这个选取的初始路线，我们根据 1）更新的交通信息，重新查找一条从起点到终点的最短路径。如果该最短路径不是原来的初始路线且经过计算该最短路径可以减少全局所有行程的耗时超过一定幅度，我们称新搜索到的这条最短路线是有效的，将原来的初始路线更新为新的最短路径。3）重复 1）2）过程直到对于每一条路线，都不存在新的有效最短路径。将该批次优化后的路线发布给用户。

4.5.2 批优化处理

为了提高算法质量，我们提出了一个批优化处理方法对由算法4-1产生的初始路线进行优化。一个初始路线组合 Π_n 包含了为最近到达的行程请求 Q_n 制定的初始路线。不同于 Π_n 的是，全局的路线组合 Π 包含了所有在当前时间之前已经规划的路线。优化处理过程重新利用了初始路线组合 Π_n ，并通过一种自感应的批处理交换策略对 Π_n 中的每一条初始路线进行优化。批优化处理的精髓在于：在每一个优化过程中，我们选择一条路线 $\pi_i \in \Pi_n$ 。我们将除了该路线的其他正在规划的初始路线作为已知的路线规划输入到交通流量信息中，重新预测未来的交通状态并尝试重新分配一条新的路线 π'_i 给行程 q_i 。我们用 π' 对原始地初始路线 $\pi \in \Pi_n$ 进行交换，该交换操作可以用 $\Pi_n.swap(\pi, \pi')$ 表示。如果该交换操作能够使所有行程的通行时间之和减少至少 $1 + \epsilon$ 的比例（e.g., $\frac{TT(\{\Pi, \Pi_n\})}{TT(\{\Pi, \Pi_n.swap(\pi, \pi')\})} > (1 + \epsilon)$ ），我

算法 4-1: InitSearch($G, q, \mathcal{L}$)

输入: 动态路网 $G = (V, E)$;
 行程请求 $q = (v_s, v_d, t)$;
 所有两点间的通行时间下界 $H(v_i, v_j)$;
 规划路线的路段标签集合 $\mathcal{L}$ 。

输出: 为请求 q 制定的初始化路线 π 。

```

1 Init:  $\forall v \in V: v.t_s = \infty, v.t_d = H(v, v_d), v.et = t$ ;  

    $v_s.pred = null, v_s.t_s = 0; \pi = \emptyset$ ;  

2  $PQ \leftarrow \{v_s\}$ ;  

3 while  $PQ \neq \emptyset$  do                                     /* 主搜索循环 */
4    $v \leftarrow PQ.pop()$ ;  

5   if  $v = v_d$  then                                       /* 到达目的节点 */
6     while  $v_d.pred \neq null$  do  

7        $\pi \leftarrow \{v_d, \pi\}; v_d \leftarrow v_d.pred$ ;  

8     return  $\pi$ ;  

9   for each edge  $e(v, v') \in G$  do                       /* 遍历邻接路段 */
10    计算当前时刻  $v.et$  下路段  $e$  的通行时间  $T(e, v.et)$ ;  

11    if  $v.t_s + T(e, v.et) \leq v'.t_s$  then  

12       $v'.t_s \leftarrow v.t_s + T(e, v.et); v'.et \leftarrow t + v'.t_s; v'.pred \leftarrow v$ ;  

13      if  $v' \notin PQ$  then  

14         $PQ.push(v')$ ;
```

们认为该操作是有效的并采用该操作去更新原有的初始路线。其中，我们采用一个极小的常数值 ϵ 排除一些“无用”的操作，这里无效操作指通过该操作只能减少很小的通行时间。此外， ϵ 的使用还保证了交换操作的次数是有限的。实际上，有效的交换操作的次数是和 ϵ 成反比的。具体来说，最大的有效交换操作的次数是 $\log_{(1+\epsilon)} \frac{TT}{TT_m}$ 。其中 TT 是当前规划的初始路线组合的总的通行时间，而 TT_m 是理想的最小通行时间。当路线组合 Q_n 中不存在有效交换操作，不能再产生一个更好的规划结果，批优化处理算法终止。在本文中，我们也提出了一个预检查策略来进一步提高效率。

4.5.2.1 交换操作

在优化过程中，除了考虑系统外车流量和系统内已经规划的路线产生的车流量外，我们进一步考虑了当前正在规划的初始路线可能对未来交通产生的潜在影响。一个新的路段标签集合 $\mathcal{L}' = \{\mathcal{L}, \mathcal{L}(\Pi_n \setminus \pi_i)\}$ 是由原有的路段标签 $\mathcal{L}$ 以及预估的正在规划的初始路线的时间信息联合组成的 (cf. 算法 4-1)。把 $\mathcal{L}'$ 作为已知的交通信息输入，我们再次执行算法4-1为行程 q_i 分配一条新的路线 π'_i 。如果该交换操

作被确定是有效的，我们将使用新路线 π'_i 替换原有的路线 $\pi_i \in \Pi_n$ 。

4.5.2.2 预检查策略

为了检查一个交换操作的有效性，我们必须更新所有路段的路段标签去计算所有行程总的通行时间的降低率，这是非常费时的。因此，我们提出了一个预检查策略进一步提高算法效率。具体来说，我们定义了一个所有行程总的通行时间的降低率上界 UB ，计算如下。

$$UB(\pi) = \frac{TT(\{\Pi, \Pi_n\})}{TT(\{\Pi, \Pi_n \setminus \pi\})} \quad (4-4)$$

注意 $TT(\{\Pi, \Pi_n \setminus \pi\}) \neq TT(\{\Pi, \Pi_n\}) - ET(\pi, t)$ ，在这里 $TT(\{\Pi, \Pi_n \setminus \pi\})$ 是将路线 π 剔除，假设该路线不存在下的所有路线的总的通行时间。我们不仅要剔除路线 π 本身的通行时间，也要消除路线 π 对其他行程的影响，因为路线 π 造成了额外的交通流量，会对其他路线的通行时间造成一定的影响。

算法 4-2: BatchRefining($G, \Pi, \Pi_n, \mathcal{L}, \epsilon$)

输入: 动态路网 $G = (V, E)$;

全局路线组合 Π ;

初始路线组合 Π_n ;

路段标签集合 $\mathcal{L}$;

参数 ϵ 。

输出: 优化后的路线组合 Π_n 。

```

1 Init:  $flag \leftarrow true$ ;
2 while  $flag$  do                                     /* 批量迭代优化 */
3      $flag \leftarrow false$ ;
4     for each route  $\pi_i \in \Pi_n$  do                   /* 遍历当前路线集合 */
5         if  $UB(\pi_i) > (1 + \epsilon)$  then
6              $\mathcal{L}' \leftarrow \{\mathcal{L}, \mathcal{L}(\Pi_n \setminus \pi_i)\}$ ;
7              $\pi' \leftarrow InitSearch(G, q_i, \mathcal{L}')$ ;
8             if 操作  $\Pi_n.swap(\pi_i, \pi')$  是有效的 then           /* 可行替换 */
9                  $\Pi_n \leftarrow \Pi_n.swap(\pi_i, \pi')$ ;  $flag \leftarrow true$ ;
    
```

4.5.2.3 算法描述

算法4-2提出了批优化处理算法的伪代码。初始地，我们使用一个标志 $flag$ 来标记在上一次“while”循环中是否存在有效的操作，算法开始我们设置其为 $true$ (line 1)。在优化过程中，对于每一条初始路线 $\pi \in \Pi_n$ ，我们检查它能否被一条新的路线 π' 替换 (lines 4–13)。具体来说，我们首先更改标志 $flag$ 为 $false$ ，假设在本轮循环中不存在有效的交换操作 (line 3)。对于每一条路线 $\pi \in \Pi_n$ ，我们通过计算提升

上界 $UB(\pi)$ 来预检查该交换操作的有效性。如果该交换操作是可能有效的，我们生成一个新的路段标签 $\mathcal{L}'$ (line 6). 将 $\mathcal{L}'$ 作为输入，我们执行算法 4-1 产生一条新的路线 π' (line 7). 如果该交换操作最终被确定是有效的，我们将采用这次交换操作并设置标志 `flag` 为 `true` 来记录当前循环仍然具有有效的交换操作 (lines 8–11). 当一轮循环，对所有的 $\pi \in \Pi_n$ 都没有检测到有效的交换操作，算法终止。也就是说已经不存在能够产生更好结果的有效操作存在。

4.5.3 CORC 问题的总体解决方案

通过整合上文提出的所有方法，我们在这里提出针对 CORC 问题的总体解决方案即 SBP 算法。SBP 算法执行流程如下：对于每一个新到达的行程请求 q ，我们先执行初始路线搜索为其分配一条初始个人路线。接着我们执行算法 4-1 周期性的批优化最近规划的初始路线，这些初始路线是为最近一段时间发布的行程请求制定的。一旦有新的行程请求发布，SBP 算法将不断执行上述操作。

4.5.3.1 优化间隔

在我们的设定中，批优化算法是以固定的时间间隔周期性执行的 (e.g., 2s)。我们用 T 表示每两次批优化处理之间的时间间隔。通过利用这样一个时间间隔 T ，我们合理考虑了更多的正在规划的初始路线对未来交通的潜在影响，在这种情况下我们能更准确地预测未来的交通状态，从而通过一次交换操作我们也许可以减少更多的通行时间，整个优化过程中总的交换次数也会减少，算法效率也就得到了提高。

算法 4-3: SBP(G, Q, T, ϵ)

输入: 动态路网 $G = (V, E)$;

行程请求流 Q ;

优化时间间隔 T ;

参数 ϵ 。

输出: 实时的最优路线组合 Π 。

1 **Init:** $\mathcal{L} \leftarrow \emptyset$; $\Pi \leftarrow \emptyset$; $\Pi_n \leftarrow \emptyset$;

2 **while** *true* **do**

/ 系统持续运行 */*

3 **for each** trip request $q \in Q$ **do**

/ 处理到达的行程请求 */*

4 $\pi \leftarrow \text{InitSearch}(G, q, \mathcal{L})$; $\Pi_n \leftarrow \{\Pi_n, \pi\}$;

5 **if** 当前时间 mod $T = 0$ **then**

/ 周期性批量优化 */*

6 $\Pi_n \leftarrow \text{BatchRefining}(G, \Pi, \Pi_n, \mathcal{L}, \epsilon)$; $\Pi \leftarrow \{\Pi, \Pi_n\}$; $\mathcal{L} \leftarrow \{\mathcal{L}, \mathcal{L}(\Pi_n)\}$;

$\Pi_n \leftarrow \emptyset$;

7 **Output** Π

// 实时输出结果

4.5.3.2 算法描述

算法 4-3 提出了针对 CORC problem 的总体解决方案，即 SBP 算法的伪代码。初始地，我们的规划系统中没有接收到任何行程请求，我们设置路段标签 $\mathcal{L}$ 和全局的路线组合 Π 都是空集。全局的路线组合 Π 记录了所有已经规划完毕的行程的路线。我们也使用了一个路线组合 Π_n 来记录为新的一批行程 Q_n 规划的初始路线，初始的 Π_n 也是空集 (line 1)。首先我们执行算法 4-1 为每一个新到达的行程 $q \in Q$ 请求分配一条初始化路线 (lines 3–6)。我们持续地为新到达的请求生成初始路线直到下一次批优化处理 (line 7)。在批优化处理过程中，我们执行算法 4-2 去生成一个优化后的路线组合 Π_n (line 8)。接下来，我们将这些优化后的路线加入全局的路线组合 Π 中，并根据新规划的路线对所有路段的路段标签记录进行更新 (lines 9–10)。每一个优化处理完成后，由于所有的初始路线已经被优化了，我们设置 Π_n 为空并继续用于记录下一批到达的行程的初始路线 (line 11)。每当我们处理完一批新的行程请求流，我们输出全局最优的路线组合 Π 作为实时的规划结果 (line 12)。

4.6 实验研究

4.6.1 实验设定

4.6.1.1 数据集

我们使用两个路网数据集用于我们的实验研究：San Joaquin County Road Network (TG 路网)^①和 the New York Road Network (NY 路网)^②。我们用邻接表进行路网图的存储。对于每个路段连边，我们基于该路段的长度分配一个数值为 20–100 间的车容量 C_e 。每个路段的最小通行时间 $T_m(e)$ 是随机生成的，位于 5–10 分钟之间。为了模拟非请求车辆，我们假设每个路段 e 上的非请求车辆的平均车流量 $\bar{x}$ 为 $0.4 \times C_e$ ，实时的非请求车辆在平均车流量附近周期性地动态变化 $0.8\bar{x}$ 到 $1.2\bar{x}$ 。给定最小的非请求车流量 $0.8\bar{x}$ 作为所有路段的静态车流量，我们提前计算好所有顶点两两间的通行时间下界，该下界即算法 4-1 中的启发值 $H(v_i, v_j)$ 。在路网较为密集的区域，我们随机采样起点-终点对来模拟行程请求。为了模拟持续到达的行程请求流，给定第一个行程的出发时间 $q_1.t$ ，接下来的第 i 个行程的出发时间 $q_i.t$ 在上一个行程请求的出发时间 $q_{i-1}.t$ 的基础上小幅度增加或保持不变。行程请求到达的速率设定见表 4-1。

① <https://www.cs.utah.edu/~lifeifei/SpatialDataset.htm>

② <https://publish.illinois.edu/dbwork/open-data>

	TG 路网	NY 路网
行程请求数 $ Q $	4,000-20,000 default 4,000	2,000-10,000 default 2,000
参数 ϵ	0.0002-0.0010 default 0.001	0.0002-0.0010 default 0.001
优化间隔 T	1s-5s default 2s	1s-5s default 2s
行程到达速率	20/s-100/s default 50/s	20/s-100/s default 50/s
α (cf. 式 4-1)	1-5 default 2	1-5 default 2
β (cf. 式 4-1)	1-3 default 2	1-3 default 2

表 4-1 参数设定

4.6.1.2 车流量的计算

在我们的实验设定中，在时刻 t 路段 e 实时的车流量 $f(e, t)$ 是由式 4-5 计算的。

$$f(e, t) = f'(e, t) + f''(e, t) \quad (4-5)$$

其中， $f'(e, t)$ 代表系统外的非请求车辆的车流量，而 $f''(e, t)$ 代表系统内请求车辆的车流量。为了模拟非请求车辆，我们假设 $f'(e, t)$ 是随时间周期性动态变化的 (e.g., $f'(e, t + T) = f'(e, t)$, $T = 2h$)。如何计算系统内请求车辆的车流量我们已经在算法 4-1 中给出。

4.6.1.3 算法评估设定

我们主要评估了自感应批处理算法 (SBP 算法) 的算法表现。为了评估该算法的结果质量，我们额外实施了一个基于个人的搜索算法 (Ind 算法)。具体来说，给定一组行程请求流 Q ，对于每一个行程请求 $q \in Q$ ，Ind 算法基于单个行程的出发时刻的交通状态生成一条最优的个人最优路线，这和一些已经存在的研究相符 [22]。由于精确算法十分耗时，在默认设定下需要至少一天的时间，因此我们不研究精确算法的表现。我们知道 SBP 算法由两部分组成：初始路线搜索和批优化处理。我们提出的初始路线搜索合理考虑了自感应的思想和网络的动态性，为了验证其优越性，我们分别又实施了 SBP*SA 算法和 SBP*DA 算法。其中，SBP*SA 算法是初始路线搜索没有采用自感知思想的 SBP 算法，即它没有考虑由我们规划系统规划的路线随交通流量造成的额外影响。SBP*DA 算法是没有考虑网络动态性的 SBP 算法，它在初始路线搜索过程中面向的是一个静态的交通状态快照即一个行程的出发时刻所对应的静态交通状态。我们使用 CPU 运行时间和所有行程总的通行时间 $TT(\Pi)$ 作为算法效率和算法准确性的评估指标。除非特别申明，我们的实验结果是超过 20 次独立实验的平均值结果。所有的算法都是在 windows10 平台上运行的，采用 Java 编程，使用 Intel(R) Core(TM) i5-9300H Processor (2.40 GHz)

的处理器和 16GB 的内存。具体的默认参数设定见表4-1。

4.6.2 实验结果

4.6.2.1 行程请求数目对算法效果的影响

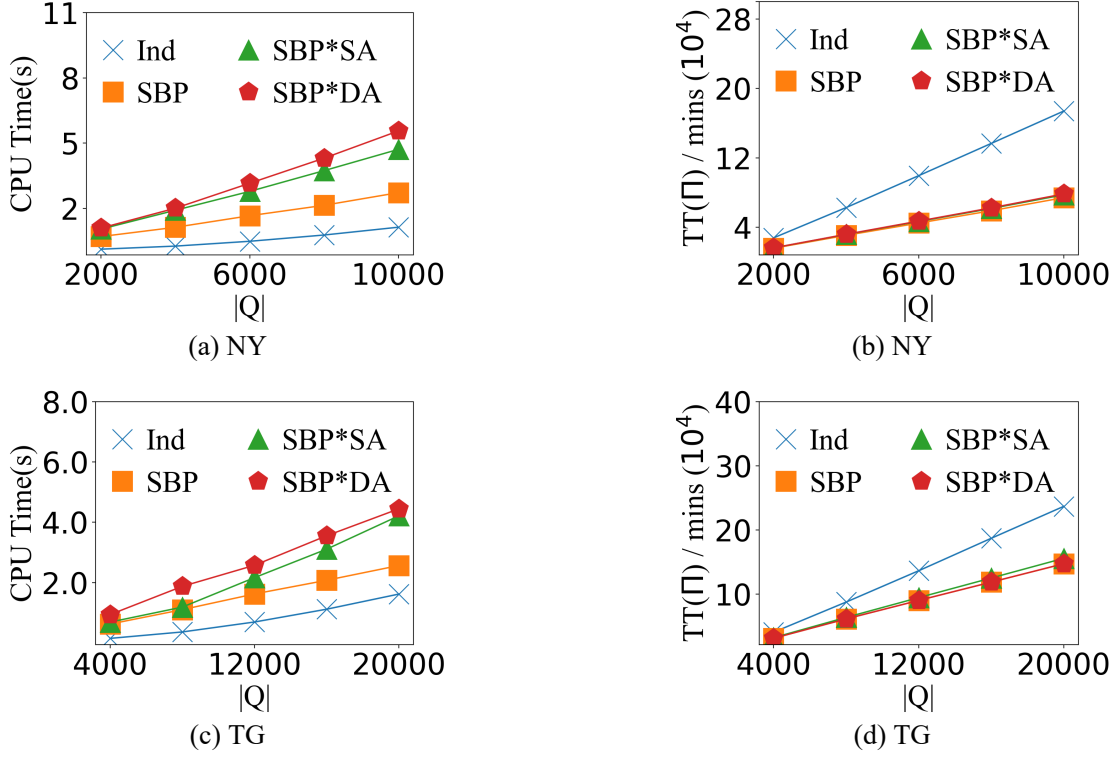


图 4-2 行程请求数目 $|Q|$ 的影响

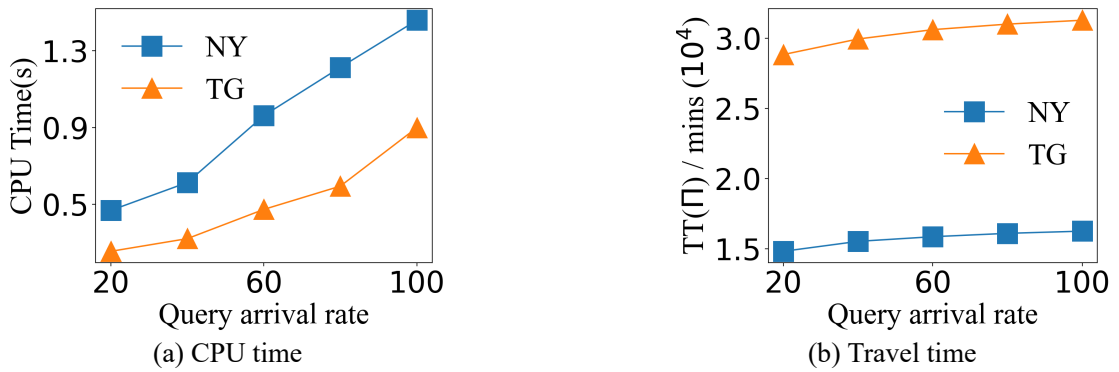


图 4-3 行程请求到达速率的影响

首先，在默认的参数设定下，我们调查了提出的算法在不同行程请求数目 $|Q|$ 下的算法表现。直觉上，行程请求数目 $|Q|$ 越大，所有行程路线的总的通行时间会增加。此外，一个更大的 $|Q|$ 也会造成更大的计算消耗，CPU 运行时间也会增加。图4-2展示了我们提出的算法在 TG 路网和 NY 路网上的表现。意料之中的，随着

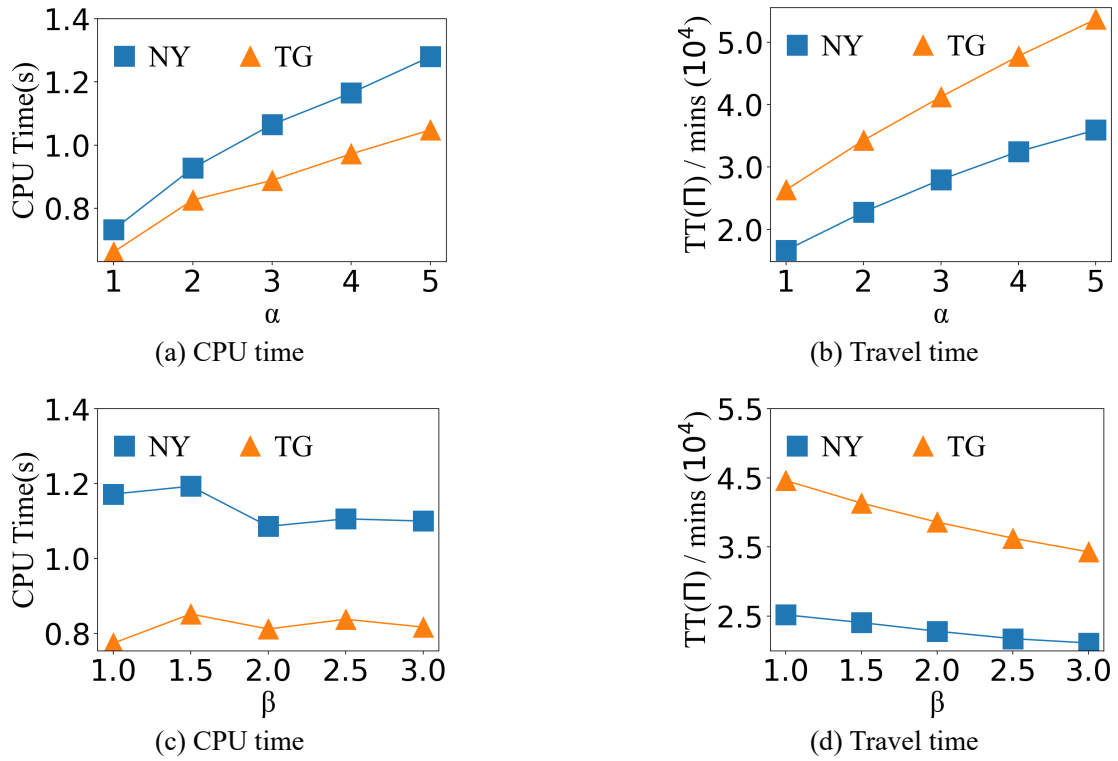


图 4-4 参数 α & β 的影响

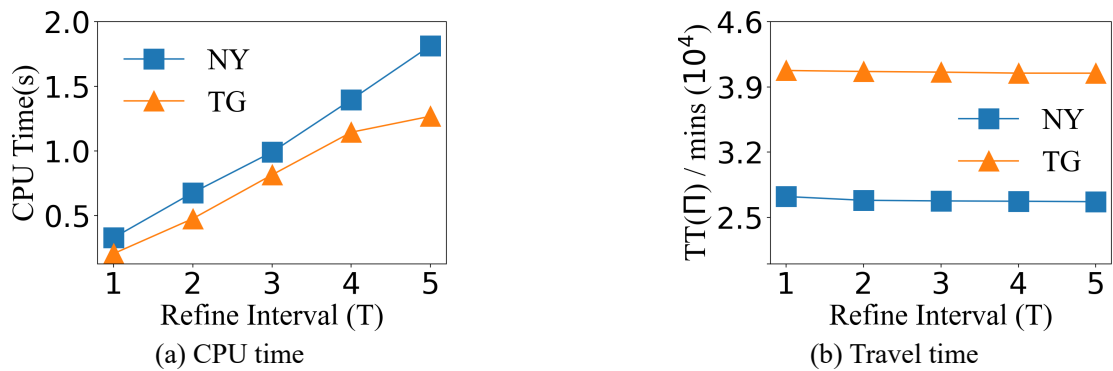


图 4-5 批优化时间间隔 T 的影响

行程请求数 $|Q|$ 的增加, 所有算法所需要的 CPU 时间都增加了。在这些算法中, 基于个人的 Ind 算法需要的 CPU 时间最小, 因为它是一次规划的, 没有经过我们所提出的批优化处理这一过程。我们也同时发现 Ind 算法产生的全局路线组合中所有路线总的通行时间 $TT(\Pi)$ 比其他算法更大, 特别是在行程请求数目较大的情况下 (e.g., 当在 NY 路网中 $|Q| = 10,000$ 和在 TG 路网中 $|Q| = 20,000$ 时, 和 Ind 算法比较, 我们的 SBP 算法能够减少超过 40% 的通行时间)。

和 SBP 算法相比, SBP*SA 和 SBP*DA 表现较差。因为它们会在优化过程中进行较多次数的交换操作, 这是非常耗时的。具体来说, SBP*SA 没有考虑到已经规划的路线也会对路段交通增加额外的车流量, 因此它在初始化路线搜索阶段进行的交通预测是不可靠的。对于 SBP*DA 算法, 它以一个行程出发时刻的交通状态快照进行路径规划。因此, SBP*SA 算法和 SBP*DA 算法所指定的初始化路线往往是低质量的, 需要在批优化过程中采取大量的交换操作来对这些低质量的初始路线进行调整, 从而产生了更多的计算消耗去提高结果质量。从图4-2(b)和图4-2(d)我们也观察到这三种算法产生的全局路线组合的所有路线的总的通行时间是非常接近的, 而且我们的 SBP 算法总能产生较小的通行时间 $TT(\Pi)$ 。这些实验结果证实了通过采用自感知的思想和考虑到网络动态性, 我们的初始路线搜索能够有效地产生高质量的处理路线, 这样的初始路线可以减少后续优化处理过程中的交换操作次数。

4.6.2.2 行程请求到达速率对算法效果的影响

图4-3展示了当行程请求到达速率发生改变时 SBP 算法的算法表现。行程请求到达速率越大, 说明单位时间会有更多的车流涌入到路网的各个路段上。在这种情况下, 每个路段车流量的增幅会更加明显。从而, 每个路段上会产生更多的额外车流量。在图4-3, 观察 CPU 时间和总的通行时间 $TT(\Pi)$, 不管是在 NY 路网还是在 TG 路网上, 我们都可以发现它们有明显的增长趋势。 $TT(\Pi)$ 增长是因为随着车流量增加, 每个路段的通行时间也相应增加了。在 TG 路网上, 所有的行程规划的路线的总的通行时间远远大于在 NY 路网上的, 这是因为在我们的默认设定中, TG 路网会有更多数目的行程请求产生。

4.6.2.3 通行时间函数各参数对算法效果的影响

图4-4展示了当我们变化通行时间函数中的参数 α 和 β (cf. 式 4-1) 时 SBP 算法的算法表现。直觉上, α 越大或者 β 越小, 各路段通行时间会增大, 从而可能会加剧交通拥堵。在图4-4(a)中, 当 α 从 1 变化到 5 时, CPU 时间和所有行程总的通行时间都有增加的趋势。相反的, 当 β 从 1 变化到 3 时, 总的通行时间具有下降的

趋势。值得注意的是，当我们变化 β 时，CPU 时间没有特别明显的变化趋势，这是因为 CPU 时间即我们所需要的计算消耗并不是和 β 的值直接相关的。

4.6.2.4 优化时间间隔对算法表现的影响

图4-5展示了当优化时间间隔 T 变化时 SBP 算法的表现。 T 越大代表着每个请求的等待时间增加了，但是规划的结果会更趋于最优，这是因为有更多的正在规划的路线被考虑到了对未来的交通状态影响中去了。在这种情况下，我们通过交换操作大大减少所有行程总的通行时间 $TT(\Pi)$ 。在图4-5(a)，CPU 时间随着 T 的增加而增加，这是因为我们在一次优化过程中需要处理更多的行程请求。在图4-5(b)中，我们可以发现 $TT(\Pi)$ 有一个轻微的减小趋势。这些结果说明了一个合适的优化时间间隔能够通过重新利用已经规划的初始路线来更准确地未来的交通状态，从而避免交通拥堵。

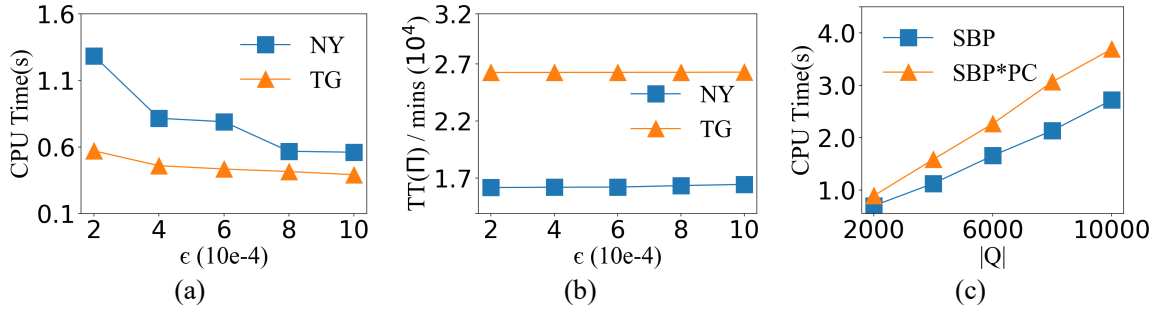


图 4-6 参数 ϵ 和预检查策略的影响 (a) CPU 运行时间；(b) 通行时间；(c) 预先检查策略的影响评估

4.6.2.5 参数 ϵ 和预检查策略对算法表现的影响

我们研究了当参数 ϵ 从 0.0002 变化到 0.001 时的 SBP 算法表现。我们知道在优化处理过程中，总的有效交换操作的次数是和参数 ϵ 的值成反比的 (cf. 算法 4-3)。 ϵ 越大，意味着有效操作的门槛越高，有效操作的次数越少，因此 CPU 时间也会随之减少。然而，一个更大的 ϵ 也会产生具有更大通行时间的全局路线组合 Π 。在图4-6(a)中，当参数 ϵ 从 0.0002 变化到 0.001 时，CPU 时间有一个减少的趋势。相反的，所有行程总的通行时间有一个轻微的增加的趋势。当 ϵ 从 0.0002 变化到 0.0004 时，所需要的 CPU 时间降幅最大 (e.g., 在 NY 路网和 TG 路网中，需要的 CPU 时间减少幅度分别超过了 36% 和 19%)。我们也发现对于 ϵ 值的选取，0.001 是非常合适的，从图中我们可以看出当 ϵ 为 0.001 时，算法的综合表现是最好的。以上实验结果说明通过利用一个合适的参数 ϵ ，SBP 算法的表现能够得到较大的提升。我们用“SBP*PC”来表示没有应用预检查策略的 SBP 算法，在图4-6(c)中，我们观察到

SBP 算法所需要的 CPU 运行时间总是比 SBP*PC 算法更少，这个实验结果说明了我们提出的预检查策略是能够进一步提升算法效率的。

4.7 本章小结

我们提出和调查了一个新颖的路线规划问题 (CORC 问题)，该问题旨在为持续的形成请求搜寻最佳的路线组合。我们提出了一个 SBP 算法来高效解决 CORC 问题。同时，一些修剪枝技术的提出也大大提升了算法效率。对应的实验研究证实了我们提出的算法能够有效地解决该问题，也证实了我们提出的剪枝技术有助于避免不必要的计算。

第五章 联合移动模式挖掘

5.1 引言

5.1.1 研究背景与意义

5.1.2 研究内容与挑战

5.1.3 研究成果与贡献

5.2 问题描述

5.2.1 路网及其构成

5.2.2 Dijkstra 算法

5.2.3 位置点间的距离计算

5.2.4 问题定义

5.3 实验研究

5.4 本章小结

第六章 时空接触搜索

6.1 引言

6.1.1 研究背景与意义

6.1.2 研究内容与挑战

6.1.3 研究成果与贡献

6.2 问题描述

6.2.1 路网及其构成

6.2.2 Dijkstra 算法

6.2.3 位置点间的距离计算

6.2.4 问题定义

6.3 实验研究

6.4 本章小结

第七章 后截断章节

致 谢

参考文献

- [1] 教育部国家语言文字工作委员. 通用规范汉字 [M]. 北京: 语文出版社, 2018.
- [2] 全国信息与文献标准化技术委员. 学位论文编写规范: GB/T 7713.1-2006[S]. 北京: 中国标准出版社, 2007: 17-20.
- [3] 王晓琰, 殷建芳, 王晓峰, 等. 关于连续出版会议论文著录格式的探讨 [J]. 学报编辑丛论, 2019, (0): 162-165.
- [4] Hu J, Zhao R, Tian M, et al. Domain decomposition method based on integral equation for solution of scattering from very thin, conducting cavity[J]. IEEE Transactions on Antennas and Propagation, 2014, 62(10): 5344-5348.
- [5] Bergamasco F, Albarelli A, Cosmo L, et al. Adopting an unconstrained ray model in light-field cameras for 3d shape reconstruction[C]. IEEE Conference on Computer Vision and Pattern Recognition, Boston, USA, 2015: 3003-3012.
- [6] Xue Q, Ji C, Ma S, et al. A survey of beam management for mmwave and thz communications towards 6g[J]. IEEE Communications Surveys & Tutorials, 2024: 1-41.
- [7] 罗杰斯. 西方文明史: 问题与源头 [M]. 潘惠霞, 魏婧, 杨艳, 等译. 2 版. 大连: 东北财经大学出版社, 2011: 1-353.
- [8] Harrington R F. Field computation by moment methods[M]. 3rd ed. New York: Wiley-IEEE Press, 1993: 76-112.
- [9] Deverell W, gler D. A companion to california history[M/OL]. New York: John Wiley & Sons, 2013: 21-22 (2013-11-15) [2014-06-24]. <http://dx.doi.org/10.1002/9781444305036>.
- [10] Clerc M. Discrete particle swarm optimization: a fuzzy combinatorial box[EB/OL]. 2010-07-16, http://clere.maurice.free.fr/ps0/Fuzzy_Discrere_PSO/Fuzzy_DPSO.html.
- [11] Audio B I D. In the spotlight: Norman rockwell museum furnishes touchscreen with directional speaker from brown innovations[EB/OL]. 2016-11-23, <https://www.brownninnovations.com/news/2016/11/23/norman-rockwell-museum-furnishes-touchscreen-with-directional-speaker-from-brown-innovations>.
- [12] Cox D A. Admiralty gulf[EB/OL]. 2015-07-24, <https://www.flickr.com/photos/navalsurfaceforces/20039065626/in/photostream>.
- [13] 陈念永. 毫米波细胞生物效应及抗肿瘤研究 [D]. 成都: 电子科技大学, 2001: 50-60.
- [14] 顾春. 牢牢把握稳中求进的总基调 [N]. 人民日报, 2012-03-31 (3).
- [15] 冯西桥. 核反应堆压力容器的 LBB 分析 [R]. 北京: 清华大学核能技术设计研究院, 1997.
- [16] 肖珍新. 一种新型排渣阀调节降温装置: ZL201120085830.0[P]. 2012-04-25.

- [17] 陈念永. 毫米波细胞生物效应及抗肿瘤研究(无页码测试)[D]. 成都: 电子科技大学, 2001.
- [18] Dai C, Zhu K, Li Z, et al. Joint decoupled multiple-association and resource allocation in full-duplex heterogeneous cellular networks: A four-sided matching game[J]. IEEE Transactions on Wireless Communications, 2022, 21(8): 6464-6477.
- [19] Khawam K, Lahoud S, El Helou M, et al. Coordinated framework for spectrum allocation and user association in 5g hetnets with mmwave[J]. IEEE Transactions on Mobile Computing, 2020, 21(4): 1226-1243.
- [20] Deng W, Liu Y, Li M, et al. Gnn-aided user association and beam selection for mmwave integrated heterogeneous networks[J]. IEEE Wireless Communications Letters, 2023, 12(11): 1836-1840.
- [21] Qiao Y, Niu Y, Han Z, et al. Joint optimization of resource allocation and user association in multi-frequency cellular networks assisted by ris[J]. IEEE Transactions on Vehicular Technology, 2024, 73(1): 826-842.
- [22] Agarwal B, Ruffini M, Muntean G M. Reduced complexity optimal resource allocation for enhanced video quality in a heterogeneous network environment[J]. IEEE Transactions on Wireless Communications, 2021, 21(5): 2892-2908.
- [23] Nwelih E. A method for real-time adaptive propagation loss modeling and estimation over los and nlos microcellular radio communication links[J]. Journal of Science and Technology Research, 2022, 4(3): 133-140.

攻读博士学位期间取得的成果

发表论文:

- [1] 李科, 陈力思, 商烁. Towards alleviating traffic congestion: Optimal route planning for massive-scale trips. *International Joint Conferences on Artificial Intelligence*, 2021, 3400-3406. (CCF A, 中科院分区, IF: 98.8)
- [2] 李科, 陈力思, 商烁, Panos Kalnis, Bin Yao. Traffic congestion alleviation over dynamic road networks: Continuous optimal route combination for trip query streams. *International Joint Conferences on Artificial Intelligence*, 2021, 62(10): 5344–5348. (CCF A, 中科院分区, IF: 98.8)
- [3] 李科, 饶漩, 庞小兵, 陈力思, 樊思琪. Route search and planning: A survey. *Big data research*, 2021, 26: 100246. (CCF A, 中科院分区, IF: 98.8)
- [4] 陈紫雯, 李科, 周偲琳, 陈力思, 商烁. Towards robust trajectory similarity computation: Representation-based spatio-temporal similarity quantification. *World Wide Web*, 26(4), 1271-1294. (CCF A, 中科院分区, IF: 98.8)
- [5] 李科, 陈力思, 商烁, 王海燕, 刘洋, Panos Kalnis, Bin Yao. Towards Controlling the Transmission of Diseases: Continuous Exposure Discovery over Massive-Scale Moving Objects. *International Joint Conferences on Artificial Intelligence*, 2022, 3891-3897. (CCF A, 中科院分区, IF: 98.8)
- [6] 李科, 王鸿宇, 陈紫雯, 陈力思. Relaxed group pattern detection over massive-scale trajectories. *Future Generation Computer Systems*, 2023, 144: 131-139. (CCF A, 中科院分区, IF: 98.8)
- [7] 王辰昊, 李科, 陈力思. Deep unified attention-based sequence modeling for online anomalous trajectory detection. *Future Generation Computer Systems*, 2023, 144: 1-11. (CCF A, 中科院分区, IF: 98.8)
- [8] 王海燕, 冯佳铭, 李科, 陈力思. Deep understanding of big geospatial data for self-driving: Data, technologies, and systems. *Future Generation Computer Systems*, 2022, 137:146-163. (CCF A, 中科院分区, IF: 98.8)
- [9] 丁重钧, 李科, 陈力思, 商烁. Parallel online similarity join over trajectory streams. *Proceedings of the ACM on Web Conference*, 2025, 3426-3437. (CCF A, 中科院分区, IF: 98.8)

- [10] 王鸿宇, 李科, 商烁. Parallel online similarity join over trajectory streams. *GeoInformatica*, 2024, 28 (2): 335-351. (CCF A, 中科院分区, IF: 98.8)
- [11] 陈紫雯, 李科, 陈力思, 胡楠, Yoshiharu Ishikawa. Co-movement aware trajectory generation via waypoint-guided generative adversarial networks. *GeoInformatica*, 2025, 29 (4):1093-1119. (CCF A, 中科院分区, IF: 98.8)
- [12] 陈紫雯, 李科, Yoshiharu Ishikawa. Ca-gen: Trajectory generation with co-movement awareness. *Proceedings of the 19th International Symposium on Spatial and Temporal Data*, 2025, 115-118. (CCF A, 中科院分区, IF: 98.8)
- [13] 李科, Leong Hou U, 商烁. App2Exa: accelerating exact kNN search via dynamic cache-guided approximation. *International Joint Conferences on Artificial Intelligence*, 2025. (CCF A, 中科院分区, IF: 98.8)
- [14] 李科, 陈力思, 商烁, Christian S Jensen, Panos Kalnis. Beyond Locations: A Motion Range-Aware Similarity Join. *In Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, 2025, 1424-1434. (CCF A, 中科院分区, IF: 98.8)

发明专利：

[1] 作者 1, 作者 2*, 作者 3, 作者 4。一种基于 xxxxx 的真气运转方法:
ZL201120846830.0. 2023-02-20.

参与项目：

- 项目号. 项目名称. 项目级别, 2020.01-2022.12.